

LAB - 3

NAME - DHIRAJ

PRN - 202501040062

DIV - CS1

BATCH - C13

Practical 3

About this unit



Practical 3

- Practice Lab Assignment**

Unit • 100% completed
- Lab Assignment**

Unit • 100% completed

3.1.1. Numpy Array Operations

18:23

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c , representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

+

numpyar...

```
1 import numpy as np
2
3 # write your code import numpy as np
4
5 # Step 1: Input rows and columns
6 r, c = map(int, input().split())
7
8 # Step 2: Input matrix elements
9 data = []
10 for _ in range(r):
11     row = list(map(int, input().split()))
12     data.append(row)
13
14 # Step 3: Handle edge case
15 if r == 0 or c == 0:
16     arr = np.empty((r, c), dtype=int)
17 else:
18     arr = np.array(data)
19
20 # Step 4: Output
21 print(arr)
22 print(arr.ndim)
23 print(arr.shape)
24 print(arr.size)
25
```

Terminal

Test cases

< Prev Reset Submit Next >



Search



ENG
IN



20:23
04-05-2026

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a . matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 print("Addition (A + B):")
13 print(matrix_a + matrix_b)
14 # Subtraction
15 print("Subtraction (A - B):")
16 print(matrix_a - matrix_b)
17 # Multiplication (element-wise)
18 print("Element-wise Multiplication (A * B):")
19 print(matrix_a * matrix_b)
20 # Matrix multiplication (dot product)
21 print("A dot B:")
22 print(np.dot(matrix_a, matrix_b))
23 # Transpose
24 print("Transpose of A:")
25 print(matrix_a.T)
```

Terminal Test cases



3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 print("Horizontal Stack:")
12 print(np.hstack((arr1,arr2)))
13
14
15 # Perform vertical stacking (vstack)
16 print("Vertical Stack:")
17 print(np.vstack((arr1,arr2)))
18
19
20
21
```

Terminal Test cases

Submit

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases



Explorer

customS...



Submit

Debugger

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9
10 sequence = np.arange(start, stop, step)
11
12 # Print the generated sequence
13 print(sequence)
```

Terminal

Test cases

< Prev

Reset

Submit

Next >



Search



ENG
IN

20:23
04-05-2026

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, ... 01:37

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A_i OR B_i).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

+

different...

1 import numpy as np

2

3 def array_operations(A, B):

4

5 # Convert A and B to NumPy array

6 A = np.array(A)

7 B = np.array(B)

8 # Arithmetic Operations

9 sum_result = A + B

10 diff_result = A - B

11 prod_result = A * B

12

13 # Statistical Operations

14 mean_A = np.mean(A)

15 median_A = np.median(A)

16 std_dev_A = np.std(A)

17

18 # Bitwise Operations

19 and_result = A & B

20 or_result = A | B

21 xor_result = A ^ B

22

23 # Output results with one space between each element

24 print("Element-wise Sum:", ' '.join(map(str, sum_result)))

25 print("Element-wise Difference:", ' '.join(map(str, diff_result)))

Terminal Test cases

Submit



Search



3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Sample Test Cases



Explorer

copyAnd...



Submit

Debugger

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13 # Modify the view
14 view_array[0] = 99
15 print("Original array after modifying view:", original_array)
16 print("View array:", view_array)
17
18 # Modify the copy
19 copy_array[1] = 88
20 print("Original array after modifying copy:", original_array)
21 print("Copy array:", copy_array)
22
```

Terminal

Test cases

< Prev

Reset

Submit

Next >



Search



ENG
IN

20:24
04-05-2026

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

00:42

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 search_indices = np.where(array1 == search_value)
13 print(search_indices[0])
14
15
16 # Count occurrences in array1
17 count_occurrences = np.count_nonzero(array1 == count_value)
18 print(count_occurrences)
19 # Broadcasting addition
20 broadcasted_array = array1 + broadcast_value
21 print(broadcasted_array)
22 # Sort the first array
23 sorted_array = np.sort(array1)
24 print(sorted_array)
```

Terminal Test cases

< Prev Reset Submit Next >



Search



ENG
IN



20:24
04-05-2026

3.2.7. Student Data Analysis and Operations

24:53

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of

Operatio...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5
6
7 print("All student Details:\n", a)
8
9 total_students = a.shape[0]
10 print("Total Students:", total_students)
11
12 roll_numbers = a[:, 0]
13 print("All Student Roll Nos", roll_numbers)
14
15 subject_1_marks = a[:, 1]
16 print("Subject 1 Marks", subject_1_marks)
17
18 min_marks_subject_2 = np.min(a[:, 2])
19 print("Min marks in Subject 2", round(min_marks_subject_2, 1))
20
21 max_marks_subject_3 = np.max(a[:, 3])
22 print("Max marks in Subject 3", round(max_marks_subject_3, 1))
23
24 print("All subject marks:", a[:, 1:])
25
```

Terminal Test cases

< Prev Reset Submit Next >



Search



ENG
IN

20:24
04-05-2026